

Klassiker 13b: Fakultät & Fibonacci

Zum Foliensatz 13b — Komplexität & Rekursion · SoSe 2026

Dorian Zwanzig

Die Aufgabe

Schreibe drei Funktionen: `fakultaet(n)` **rekursiv** (Basisfall $0! = 1$, rekursiver Fall $n! = n \cdot (n - 1)!$), `fib_rekursiv(n)` (Fibonacci mit zwei Selbstaufrufen) und `fib_iterativ(n)` (dieselbe Folge mit einer Schleife). Das Programm gibt `4!` aus und vergleicht dann beide Fibonacci-Varianten für $n = 10$ und $n = 30$. Bei $n = 30$ sollst du den Unterschied **spüren**: Die rekursive Variante braucht spürbar lange ($O(2^n)$), die iterative antwortet sofort ($O(n)$).

Erwartetes Verhalten

```
4! = 24
fib(10) rekursiv:  55
fib(10) iterativ:  55
fib(30) rekursiv:  832040  (spuerbar langsam)
fib(30) iterativ:  832040  (sofort)
```

Erlaubte Bausteine

Alles bis Notebook 13b — mehr brauchst du nicht: `print()` und f-Strings · `def / return` · `if / else` · Vergleiche (`==`, `<`) · `for` mit `range()` · Tupel-Tausch `a, b = b, a + b` · **neu: Rekursion** (Basisfall + rekursiver Fall)

Gestufte Hinweise

Erst selbst probieren — dann Hinweis für Hinweis aufdecken.

Hinweis 1 (Ansatz): Schreibe bei jeder rekursiven Funktion den **Basisfall zuerst** — dann den rekursiven Fall, der sich mit einem **strikt kleineren** n selbst aufruft. Sonst droht ein `RecursionError`.

Hinweis 2 (Struktur): Pseudocode:

```
fakultaet(n):
    wenn n == 0: gib 1 zurueck          # Basisfall
    sonst: gib n * fakultaet(n - 1) zurueck

fib_iterativ(n):
    a = 0, b = 1
    n-mal wiederholen: a, b = b, a + b
    gib a zurueck
```

Hinweis 3 (der Kniff): `fib_rekursiv` braucht **zwei** Basisfälle — `if n < 2: return n` erledigt beide auf einmal (`fib(0) = 0`, `fib(1) = 1`). Danach: `return fib_rekursiv(n - 1) + fib_rekursiv(n - 2)`.

Bonus

Notbremse testen: Rufe `fakultaet(3000)` auf — Python bricht mit `RecursionError` ab, sobald ca. 1000 Aufrufe gleichzeitig offen sind. Warum passiert das bei `fib_iterativ` nie? **Türme von Hanoi:** der berühmteste Rekursions-Klassiker wartet als Bonus-Übung im Notebook 13b.

Musterlösung nach der Vorlesung:

Praesentationen_src/13b_Algorithmen_Grundlagen/90_klassiker_fakultaet_fibonacci.py

Vertiefung: Notebook 13b, Kapitel Rekursion.